第七章 优化、API化与文档：从技能到可交付产品

> **本章导言**
>
> 经过前六章的学习与开发，你已经将一个经典的设计研究方法成功重构为可运行的AI技能（Skill）。这个Skill通过了系统测试，在准确性、一致性、效率、健壮性和体验等维度上都有了量化的验证。然而，一个通过了测试的“课程项目”与一个可以交付给用户使用的“产品”之间，仍然存在若干关键的差距：代码是否整洁、文档是否清晰、核心能力是否可以被其他程序调用、演示是否能够有效传达价值。
>
> 本章聚焦于弥合这些差距。我们将首先学习如何将Skill中最稳定、最独立的子功能封装为API，使用FastAPI框架构建可被外部程序调用的服务端点。随后，我们将基于测试报告中的发现，对Skill进行最后一轮关键修复——不是追求完美，而是有选择地解决最影响用户体验的痛点。在此基础上，我们将撰写专业的技术文档（README）、策划演示视频脚本，并完成所有交付物的整合与代码仓库的整理。本章结束时，你将拥有一个可以被任何人理解、使用和集成的完整工具包——这是从“学会开发”到“学会交付”的最后一跃。

本章学习目标

完成本章学习后，读者应能够：

1. 从现有Skill中识别适合封装为API的子功能，并应用四个筛选标准进行评估；
2. 使用FastAPI框架和Vibe Coding将分析功能封装为RESTful API端点，包含请求/响应模型、错误处理和自动文档；
3. 编写API客户端调用示例和自动化测试脚本，验证端点的正确性和健壮性；
4. 基于测试报告中的发现，对Skill进行优先级驱动的关键问题修复；
5. 使用AI辅助生成专业的中文README文档和演示视频脚本；
6. 整理GitHub代码仓库，完成最终交付物的整合与版本标记。

7.1 从Skill到API：设计理念与候选识别

7.1.1 为什么要将Skill封装为API

在前六章中，我们始终围绕“Skill”这一核心概念展开——一个面向人类用户的、带有图形界面的、有人工修正节点的交互式工具。这种形态对于设计研究者本人或同组协作非常友好：用户可以看到进度、可以修改AI的输出、可以导出报告。然而，这种形态也天然限制了能力的复用范围：如果你希望其他工具或工作流能够自动调用你的分析能力（例如，在一个用户旅程地图生成器中自动标注每个触点的情绪数据），Skill的图形界面就成了障碍。

API（应用程序编程接口）正是为解决这一问题而生的。将Skill中的核心子功能封装为API，意味着：

- **可嵌入性**：其他设计工具（如Figma插件、Miro小工具、Slack机器人）可以通过HTTP请求调用你的分析能力，将其嵌入到更大的工作流中。
- **可组合性**：多个API可以像乐高积木一样自由组合。例如，访谈摘要API + Persona生成API + 价值主张对齐API，可以串联成一条自动化的用户研究到策略生成的流水线。
- **可规模化**：一个API可以被成百上千次调用而无需人工干预每个实例，适合集成到CI/CD流程或持续监控系统中。

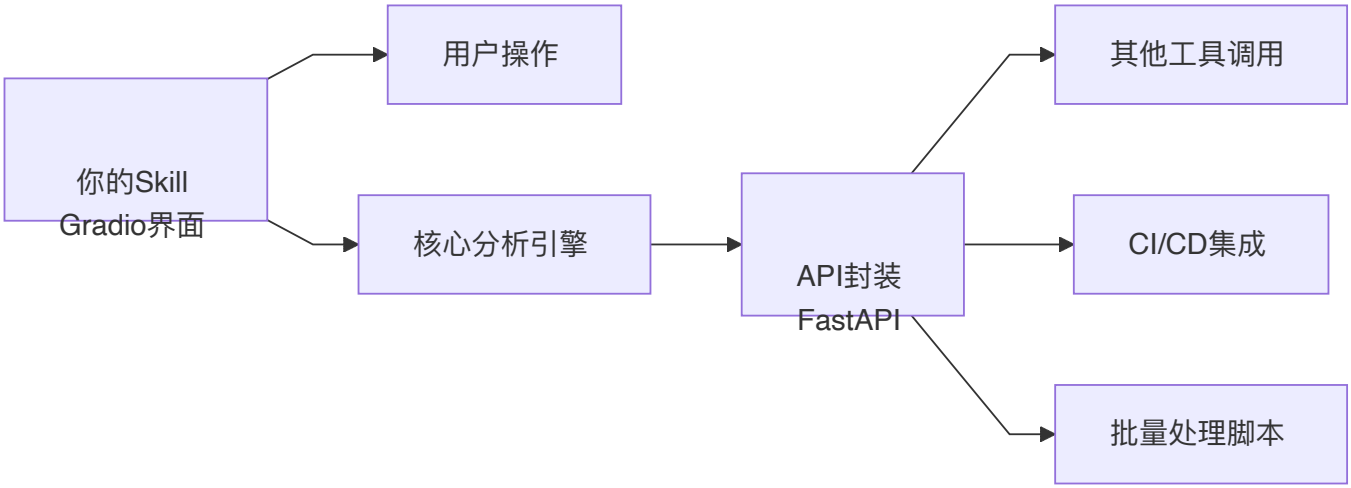


图7-1 从Skill到API的架构演进

7.1.2 API候选的四个筛选标准

并非Skill中的所有功能都适合封装为API。初学者常见的误解是“把我的整个Skill做成一个API”——但Skill包含界面、多步处理链、人工修正节点、输出格式化等，这些紧密依赖人类交互的部分一旦脱离GUI，接口就会变得极其复杂，难以被外部程序使用。

一个合适的API候选应满足以下四个标准中的至少三个。让我们逐一审视。

标准一：输入输出明确且稳定。

API端点的输入和输出必须能被清晰定义、无歧义。输入是字符串还是整数？输出是一个标签还是一段文本还是一组数值？这些必须毫不含糊。

✅ **正例**：情感分析子功能——输入是一段文本字符串，输出是一个包含`sentiment`（正面/负面/中性）、`intensity`（1-10）和`confidence`（0-1）的JSON对象。这个接口在任何上下文中都是明确的。

❌ **反例**：Skill中的“交互式主题确认”环节——输入是“用户在界面上拖拽调整了聚类结果”，输出是“更新后的主题结构”。这个输入输出边界在GUI上下文中是清晰的（因为用户操作本身定义了输入），但一旦脱离GUI，就没有明确的接口形态。这类依赖GUI交互的功能不适合封装为API。

标准二：功能独立，不依赖前序步骤的大量上下文。

一个好的API端点应该能独立完成其任务，而不需要调用者先执行一套复杂的预处理流程。如果调用你的端点之前，必须先用另一个端点处理数据、再手动提取某个字段、再作为参数传入——这个调用链太脆弱了。

API端点之间可以有依赖关系（多个端点可以被调用者串联使用），但要确保每个端点在被单独调用时，只需要调用者提供该端点自身处理需要的直接输入。调用者不需要理解上一环节的内部处理细节。

标准三：调用频率可能较高。

如果某个子功能在你的Skill中每次只被调用一次，而且整个Skill本身也不是一个高频使用的工具，那将其封装为API的收益有限。反之，如果一个子功能在理论上有被频繁调用的潜力——例如情感分析可以被反复应用于大批量文本，或启发式原则审查可以被嵌入CI/CD每次自动扫描——封装为API的价值就更大。

标准四：性能可预测。

API调用者期望响应时间是可预测的。如果一个子功能的处理时间波动极大（如某次聚类可能耗时2秒，另一次因数据规模大而耗时30秒），封装为API时需要明确标注预期的处理时间和数据量限制。一般来说，同步API端点的响应时间应控制在数秒以内；对于更长时间的处理，应采用异步模式（提交任务立即返回任务ID → 稍后通过另一个端点查询结果）。

7.1.3 典型方法类型的API候选示例

为了帮助读者快速定位自己Skill中的API候选，表7-1按设计研究方法类型给出了常见示例。

方法类型	候选API端点	输入	输出
访谈分析	`/api/sentiment-analysis`	文本	情感极性、强度、关键短语
	`/api/topic-extraction`	文本列表	聚类主题及代表性短语
	`/api/interview-summary`	完整访谈文本	指定长度摘要
亲和图/主题分析	`/api/text-clustering`	文本条目列表	分组结构及标签建议
	`/api/label-suggestion`	一组文本条目	该组的命名建议
	`/api/semantic-similarity`	两段文本	相似度分数
问卷/Persona	`/api/user-segmentation`	用户数据表格	聚类分群结果
	`/api/generate-persona`	分群统计数据	Persona卡片
	`/api/open-coding`	开放题回复列表	编码建议
可用性测试	`/api/heuristic-review`	界面截图 + 原则列表	违规点清单
	`/api/usability-issues`	用户操作日志	问题时刻列表
	`/api/task-segmentation`	操作序列	自动切分的任务段
趋势扫描/SWOT	`/api/signal-detect`	关键词 + 时间范围	异常信号检测结果
	`/api/evidence-extract`	文本 + 象限描述	归类的证据片段
	`/api/trend-briefing`	趋势信号列表	自然语言简报

表7-1 不同设计研究方法类型的API候选示例

7.1.4 从测试报告中选择API候选

在确定API候选时，测试报告中的数据可以为你提供重要的决策依据。重点关注测试中准确性最高、稳定性最好、耗时最可预测的子功能——这些是封装为API后最有可能被外部开发者信赖和使用的候选。

课堂练习：根据你的Skill设计文档和测试报告，识别2-3个API候选，并为每个候选填写以下表格：

字段	内容
端点名称	`/api/xxx`
功能描述	一句话说明
输入参数	名称、类型、是否必需、取值范围
输出结构	JSON结构、各字段含义
调用场景	谁会调用？在什么情境下？
筛选标准满足度	明确输入输出？独立？高频？性能可预测？

7.2 FastAPI基础：用自然语言构建API

7.2.1 为什么选择FastAPI

FastAPI是一个用于构建Python API的现代框架。它的名字揭示了它的两个核心特点：**Fast**（快）——无论是运行速度还是开发速度；以及**API**——专注于API构建。对于本课程而言，FastAPI有三个特别适合Vibe Coding的特性：

第一，代码结构高度直观。定义一个API端点就像写一个普通的Python函数：用装饰器标注HTTP方法和路径，函数参数自动解析为请求参数，函数返回值自动序列化为JSON响应。这种"声明式"的风格使得AI生成FastAPI代码的质量通常很高——因为模式简单且训练数据中大量存在。

第二，自动生成交互式文档。写好端点后，FastAPI会自动生成一个`/docs`页面（基于Swagger UI），以网页形式展示所有端点、参数模型、响应模型，甚至允许在页面中直接填写参数并"试用"API。这意味着你的API天然自带一份无需额外编写的技术文档。

第三，内置数据验证。通过Pydantic模型定义请求和响应的数据结构，FastAPI会在运行时自动校验数据类型、必填字段、取值范围等，并在校验失败时返回清晰的错误信息。这大大减少了手动编写参数校验代码的工作量。

7.2.2 一个最简FastAPI示例

在进入Vibe Coding之前，我们先理解一个最简示例的结构。**请注意**：在实际开发中，你不需要手动编写这些代码——将需求描述给AI即可生成。此处仅用于建立概念理解。

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="情感分析API")

# 定义请求模型
class AnalysisRequest(BaseModel):
    text: str  # 需要分析的文本，必需

# 定义响应模型
class AnalysisResponse(BaseModel):
    sentiment: str      # "正面"/"负面"/"中性"
    confidence: float   # 0-1之间的置信度
```

```python

@app.post("/api/sentiment-analysis", response_model=AnalysisResponse)
def analyze_sentiment(request: AnalysisRequest):
    # 这里调用实际的AI分析逻辑
    result = call_llm_for_sentiment(request.text)
    return result
```

将这个文件保存为`api.py`，运行`uvicorn api:app --reload`，然后打开浏览器访问`http://localhost:8000/docs`——你会看到一个完整的交互式API文档页面。输入测试文本，点击“Execute”，即可看到返回的JSON结果。

7.2.3 Vibe Coding封装API的核心提示词模板

以下的提示词模板可以直接用于让AI生成一个完整的FastAPI应用。请将方括号中的内容替换为你的实际需求。

```
我有一个分析函数 [函数名]([输入参数])，它接收 [输入描述]，调用 [AI模型名称] API，返回 [输出描述]。
输出结构为：{字段1：类型，字段2：类型，...}。

请将这个功能封装为一个FastAPI应用，具体要求：

1. **API端点**：
   - POST /api/[端点名]：接收 [输入]，返回分析结果
   - GET /api/health：返回 {"status": "ok", "version": "1.0.0"}

2. **请求模型**（Pydantic）：
   - [参数名]（[类型]，必需/可选）：[描述]，取值范围 [如1-2000字符]

3. **响应模型**（Pydantic）：
   - 包含上面输出结构的所有字段
   - 额外增加 request_id（str）和 processing_time（float）

4. **错误处理**：
   - 输入验证失败（空值、超长、类型错误）→ 返回422和统一错误格式
   - 内部AI调用失败（超时、限流、返回格式异常）→ 返回503和可读错误信息
   - 统一错误格式：{"error": {"code": "...", "message": "..."}}

5. **CORS设置**：允许所有来源的跨域请求（开发阶段）

6. **配置管理**：使用pydantic-settings从.env文件读取API Key、模型名称等

7. **日志记录**：每次请求记录时间戳、request_id、处理耗时、状态码到控制台和api.log

8. **速率限制**（可选）：对主端点设置每分钟最多30次请求

请生成完整的Python代码，并告诉我如何启动服务。
```

7.2.4 从Skill代码中提取独立分析模块

在封装API时，一个常见的技术挑战是：你的分析函数目前深嵌在Skill的主处理链中，可能直接依赖了状态字典、Gradio上下文或其他UI组件。要解决这个问题，你需要先重构代码，将核心分析逻辑提取为独立的、可导入的模块。

Vibe Coding重构提示词：

```
我当前的Skill代码中，分析逻辑分散在主处理链的 [函数名] 函数里，这个函数内部依赖了状态字典和全局配置。请帮我：

1. 从现有代码中提取纯分析逻辑—只接收必要的输入参数，调用AI模型，返回分析结果。
2. 将提取出的逻辑放入一个独立的 `[模块名]_engine.py` 文件中的 `[模块名]Engine` 类。
3. 这个类不依赖任何Gradio组件、状态字典或UI相关代码。所有配置（如API Key、模型名称）通过构造函数参数或环境变量传入。
4. 在我的Skill主代码中，导入并调用这个Engine类，保持Skill功能不变。
5. API端点的代码也导入同一个Engine类，避免代码重复。

提取后，请提供独立的测试代码片段，验证Engine类可以不依赖Skill主程序独立运行。
```

提取完成后，你应该能够在一个新的Python文件中执行类似以下代码来验证独立性：

```python
from sentiment_engine import SentimentEngine

engine = SentimentEngine()  # 从环境变量读取配置
result = engine.analyze("这个产品的界面设计非常直观")
print(result)  # {'sentiment': '正面', 'intensity': 8, 'confidence': 0.95}
```

7.2.5 API的错误处理与响应质量

在GUI中，一些输出质量瑕疵可以通过用户的交互来补偿——用户可以编辑AI的输出、忽略不合理的结果。但在API中，调用者没有这个交互机会。API返回的结果将被程序化地消费和使用。因此，API的输出质量需要比GUI中的输出更稳定、更精确。

优化方向一：降低随机性。在API端点中调用大模型时，将`temperature`参数设置为0或接近0的值。API的调用者期望可预测的、可复现的结果。

优化方向二：强化输出格式约束。在API的分析提示词中，比GUI版本更严格地要求输出格式。例如："只返回JSON，不要添加任何解释文字"——因为在API中，多一行解释文字就可能导致调用者的JSON解析失败。

优化方向三：增加输出完整性校验。在API端点函数中，发送响应之前增加一个校验步骤：检查返回的JSON是否包含所有必需字段、字段类型是否正确、数值是否在预期范围内。如果校验失败，记录错误日志并返回503，而不是把不完整的响应发给调用者。

> 请在API端点的处理函数中，在返回响应之前增加一个校验步骤。检查返回的分析结果中所有必需字段是否存在且类型正确。如果校验失败（如情感标签不是三个预期值之一），记录完整的错误详情到日志，并返回503状态码，同时返回一个包含error_code和message的JSON，而不是把不可靠的结果返回给调用者。

7.3 API测试与客户端调用示例

7.3.1 自动化测试脚本

API开发完成后，需要一套自动化的测试脚本来验证其正确性和健壮性。这些脚本应该覆盖：

- **正常场景**：预期输入下，响应状态码200，返回结构符合预期。
- **边界场景**：极短/极长输入、接近限制的数值。
- **异常场景**：空输入、类型错误、超长输入——验证返回4xx而非崩溃。
- **性能场景**：连续多次请求，记录响应时间和成功率。

Vibe Coding生成测试脚本的提示词：

```
请写一个Python测试脚本 `test_api.py`，用于测试我的 [API端点名] 端点。

API基础URL：http://localhost:8000

测试用例包括：

1. **正常场景**（3条）：
   - 明显正面文本、明显负面文本、中性文本各一条
   - 验证：状态码200，响应中包含所有必需字段（sentiment, intensity, confidence等）

2. **边界场景**（2条）：
   - 非常短的文本（如"好"）
   - 接近长度限制的文本（如1900字符，如果限制是2000）

3. **异常场景**（2条）：
   - 空字符串 → 预期状态码422
   - 超长文本（超过限制） → 预期状态码422

4. **性能场景**（1条）：
   - 连续发送5个相同的正常请求，记录每个的响应时间，计算平均值和标准差

脚本要求：
- 使用requests库
- 每个测试用例有明确的名称和预期结果
- 通过打印绿色[PASS]，失败打印红色[FAIL]并附带失败原因
- 最后汇总：总测试数、通过数、失败数、通过率
- 如果API不可达（连接失败），给出清晰提示而非直接崩溃
```

7.3.2 客户端调用示例

为了让其他开发者能够快速上手使用你的API，你需要提供一个简洁的客户端调用示例。这个示例应该可以直接复制粘贴到Python环境中运行。

Vibe Coding生成客户端示例：

```
请基于我的API端点文档，生成一个Python客户端调用示例 `client_example.py`。要求：

1. 导入requests库
2. 定义API基础URL（使用变量，方便修改）
3. 构造一个示例请求（包含典型输入）
4. 发送POST请求
5. 检查响应状态码，如果成功则打印格式化的结果（如用json.dumps(indent=2)）
6. 如果失败，打印错误信息和建议操作（如“请检查API服务是否运行在8000端口”）
7. 代码中包含清晰的中文注释

输出为完整的Python脚本。
```

生成的示例代码大致如下：

```python
import requests
import json

# API端点URL（根据实际部署地址修改）
API_URL = "http://localhost:8000/api/sentiment-analysis"

# 示例输入
sample_text = "这个产品的界面设计非常直观，但是加载速度有点慢。"

# 发送请求
response = requests.post(API_URL, json={"text": sample_text})

if response.status_code == 200:
    result = response.json()
    print("分析成功！")
    print(json.dumps(result, indent=2, ensure_ascii=False))
else:
    print(f"请求失败，状态码：{response.status_code}")
    print(f"错误信息：{response.text}")
    print("请确认API服务已启动（uvicorn api:app --reload）")
```

7.3.3 通过Swagger UI手动测试

FastAPI自动生成的`/docs`页面是最好的交互式测试工具。在API服务运行后，访问`http://localhost:8000/docs`，你会看到一个网页，其中：

- 列出了所有API端点
- 每个端点有“Try it out”按钮，点击后可以填写参数
- 填写后点击“Execute”，页面会显示请求的curl命令和响应结果

这个页面本身就是API文档的一部分，也是你向他人展示API功能的最直观方式。

7.4 基于测试报告的关键修复

7.4.1 修复优先级管理

在第6章中，你完成了系统测试并撰写了一份测试报告。报告中的“改进计划”章节应该列出了按优先级排序的改进项（P0/P1/P2）。在第7周的有限时间内，你不可能修复所有问题。你需要做出选择：聚焦于那些“修复成本低、用户体验影响大”的问题。

优先级决策矩阵：

影响程度 \ 修复成本	低成本	高成本
高影响	✅ 立即修复（P0）	⚠️ 评估必要性，或列入未来计划（P1）
低影响	🔷 有余力时修复（P2）	❌ 暂不处理，文档中标注已知限制

以下是一些典型问题的修复方法与Vibe Coding描述：

问题类型一：准确性相关的提示词优化

如果测试报告显示某类数据上的分析准确率明显偏低（如Skill对委婉表达的判断准确率只有50%），可以通过优化提示词来改善。

> 在我的Skill的分析提示词中，系统性地漏判了委婉的负面表达。请修改提示词，在情感判断的部分增加以下指导：
>
> 1. 明确列出中文中常见的委婉负面表达模式（如“还可以但...”“怎么说呢...”“也不是不行”），并给出它们为何被判断为负面的分析逻辑
> 2. 在提示词中增加3个少样本示例——每个示例展示一段文本和它对应的正确判断（情感极性+判断依据）
> 3. 要求模型在遇到不确定的表达时降低置信度标注，而不是猜测一个确定标签

问题类型二：交互体验相关的Gradio改进

如果中期评审或体验测试中暴露了关键的交互问题——如“用户找不到下载按钮”“人工修正节点的操作方式令人困惑”——可以针对性地修改Gradio布局或交互逻辑。

> 测试反馈显示，用户在使用我的亲和图Skill时，不知道可以在右侧编辑分组名称。请修改界面：
>
> 1. 在每个分组名称旁边添加一个“✎ 编辑”图标，光标悬停时显示“双击编辑分组名称”
> 2. 编辑完成后按Enter键确认，显示一个绿色的✓标记持续2秒
> 3. 在界面顶部添加一个提示横幅：“提示：双击任何分组名称即可修改”

问题类型三：健壮性相关的异常处理

如果测试中发现某些异常输入导致Skill崩溃或静默错误，这些是P0级别的修复项。一个会在用户上传错误格式文件时崩溃的工具，给人的第一印象比分析不够准确更致命。

> 测试发现，当用户上传的文件编码不是UTF-8时，我的Skill会直接崩溃（UnicodeDecodeError）。请修复：
>
> 1. 在文件读取时，尝试多种编码（UTF-8, GBK, Latin-1）
> 2. 使用chardet库自动检测文件编码作为后备方案
> 3. 如果所有编码都失败，在界面上显示友好的错误提示：“文件编码无法识别，请确保文件为UTF-8或

> GBK编码”
> 4. 不要向用户暴露Python的堆栈跟踪信息

7.4.2 修复完成后更新文档

任何修复都应该反映在文档中。如果修复了测试报告中的某个问题，在README的“已知限制”章节中删除或更新相关条目。如果修复涉及API行为的变化（如输出格式调整），务必更新API文档和客户端示例。

7.5 技术文档的撰写：README

7.5.1 README的重要性

README是项目的“门面”。如果你的代码仓库发布在GitHub上，README决定了路过的人是否会停下来了解你的项目。对于课程而言，README是导师和评审者理解你工作的入口文档。一份好的README应该在3分钟内让读者理解：这个工具做什么、为什么有用、怎么用。

7.5.2 标准README结构

一份专业的项目README通常包含以下10个部分。对于课程项目，前7项为必需，后3项可根据实际情况决定是否包含。

```
# [工具名称] — [一句简短描述]

![Python Version](https://img.shields.io/badge/python-3.10+-blue.svg)
![License](https://img.shields.io/badge/license-MIT-green.svg)

## 📖 简介
用2-3句话说明工具的定位、解决什么问题、目标用户是谁。

## ✨ 核心功能
- 功能1：简要描述
- 功能2：简要描述
- 功能3：简要描述

## 🚀 快速开始

### 前置要求
- Python 3.10+
- DeepSeek API Key（或其他模型API Key）

### 安装
```bash
git clone [仓库URL]
cd [项目目录]
pip install -r requirements.txt
```

配置
```

1. 复制 `.env.example` 为 `.env`
2. 填入你的API Key

运行Gradio界面

```
python app.py
```

运行API服务

```
uvicorn api:app --reload
```

访问 http://localhost:8000/docs 查看交互式API文档。

📝 使用说明

简述主要操作流程（2-3步）。

🔌 API文档

- `POST /api/[端点名]`：[功能描述]
- 请求示例：`{"text": "..."}`
- 响应示例：`{"sentiment": "...", "confidence": 0.95}`

详细API文档请访问 `/docs` 端点。

📁 项目结构

```
├── app.py              # Gradio界面主程序
├── api.py              # FastAPI服务
├── sentiment_engine.py # 核心分析引擎
├── requirements.txt    # 依赖列表
├── .env.example        # 环境变量模板
└── README.md
```

👥 开发者

- [姓名1] - [角色]
- [姓名2] - [角色]
- 本作品为《智能设计系统》课程项目

⚠️ 已知限制

- 限制1：描述
- 限制2：描述

🔮 未来计划

- 计划1
- 计划2

🙏 致谢

- 使用DeepSeek API进行文本分析
- 基于Gradio和FastAPI构建

```

### 7.5.3 用AI生成README初稿

> 请基于我的项目代码和以下信息生成一份专业的中文README.md。
>
> **项目名称**：[Skill名称]
> **一句话描述**：[如“用户访谈文本的智能主题分析工具”]
> **核心功能**：[列出3-5个]
> **技术栈**：Gradio, FastAPI, DeepSeek API, sentence-transformers
> **项目结构**：[粘贴目录树或简要说明]
>
> 请按照上述10部分结构生成README初稿。在“快速开始”部分，提供可以直接复制粘贴的安装和运行命令。在“已知限制”部分，基于我测试报告中的发现列出2-3个限制。语言风格专业但不刻板，面向设计研究者和开发者。

### 7.5.4 README的审核要点

AI生成初稿后，你需要逐项审核以下内容：

- **准确性**：安装步骤在你的电脑上真的能跑通吗？如果复制到一个干净环境，能成功启动吗？
- **一致性**：README中描述的端口号、文件路径、命令与你的实际项目一致吗？（AI可能在不同地方生成不一致的端口号）
- **完整性**：API端点列表是否完整？每个端点的输入输出描述是否准确？
- **可读性**：一个对你的项目一无所知的人，读完“简介”和“快速开始”后，能不能在3分钟内启动你的工具？

---

## 7.6 演示视频脚本策划

### 7.6.1 演示视频的定位

在第8周的最终课程展示中，每组有5分钟时间进行总结陈述，其中包含一段3分钟的录制演示视频。这3分钟是展示你六周工作成果的浓缩窗口。一个好的演示视频能让评审者在3分钟内理解：你解决了什么问题、你的工具怎么用、它的亮点是什么。

### 7.6.2 三段式叙事结构

经典的演示视频遵循“问题-方案-价值”的三段式结构：

**第一段（约30秒）：问题与动机**

```

不先介绍工具，先让观众感受到痛点。用一个具体的场景开场：

> “设计研究者在进行[方法名]时，最耗时的环节是什么？[举一个具体数字，如分析一场45分钟的访谈需要3小时]。今天，我们想看看AI能把这个时间缩短到多少。”

第二段（约2分钟）：工具演示

这是核心部分。用一个连贯的场景从头到尾展示工具的核心流程。建议的镜头序列：

1. **输入**（10秒）：上传数据文件或粘贴文本
2. **处理**（15秒）：展示进度条或阶段性提示（可倍速）
3. **AI分析结果**（20秒）：展示初始输出（卡片、聚类、报告）
4. **人工修正**（30秒）：**重点**——展示某个AI判断不够准确的地方，用户如何修改（编辑/拖拽/驳回），以及修改后系统的响应
5. **最终输出**（15秒）：展示整理好的报告或导出结果

第三段（约30秒）：总结与价值

用一组对比数据收尾：

> “[Skill名称]将[方法名]的分析时间从[传统耗时]缩短到[实测耗时]。同时，关键判断权始终保留在研究者手中。欢迎访问我们的GitHub仓库体验。”

7.6.3 用AI生成分镜脚本

> 我需要为我的设计研究AI Skill录制一个3分钟的演示视频。请帮我策划一份分镜脚本。
>
> **我的Skill**：[一句话描述]
> **核心功能**：[列出2-3个最想展示的]
> **目标观众**：设计研究课程的师生和潜在用户
>
> 请按照以下结构生成脚本，使用表格格式：
>
> | 镜头 | 时间 | 画面内容 | 旁白 | 备注 |
> |---|---|---|---|---|
> | 1 | 0:00-0:10 | [开场画面] | [旁白] | |
> | ... | ... | ... | ... | |
> | 结束 | 2:50-3:00 | [结束画面] | [总结语] | |
>
> 要求在人工修正环节展示“人机协同”的特色——例如，AI的建议与用户的专业知识如何结合。总时长严格控制在3分钟。

7.6.4 演示视频的录制与剪辑

录制建议：
- 使用OBS Studio（免费开源）或系统自带的屏幕录制工具
- 分辨率至少1920×1080
- 旁白可以使用麦克风录制，也可以后期配音
- 如果处理过程中有较长的等待（如API调用），可以录制后剪掉等待部分，用文字说明“处理中...”
- 最终输出MP4格式，文件大小控制在100MB以内

7.7 最终交付物整合与GitHub仓库整理

7.7.1 完整交付物清单

在最终提交前，请对照表7-2逐项确认。

序号	交付物	验收标准	状态
1	Gradio Skill	在另一台电脑上按README能运行并完成分析	□
2	FastAPI端点	`/docs`可访问，至少一个端点能正确响应	□
3	API测试脚本	所有测试用例通过（或未通过的有文档说明）	□
4	客户端调用示例	可复制粘贴运行的Python脚本	□
5	测试报告	包含五维评估和改进计划	□
6	README	包含快速开始、API文档、已知限制	□
7	Vibe Coding日志	覆盖第4-7周主要开发对话	□
8	演示视频	3分钟以内，三段式叙事	□
9	个人反思论文	800-1200字	□
10	GitHub仓库	上述所有文件，无敏感信息	□

表7-2 最终交付物清单

7.7.2 Git仓库整理

对于不熟悉Git的读者，可以使用以下提示词获得操作指引：

> 我想把我的项目上传到GitHub。请告诉我每一步需要做什么，包括：
> - 初始化Git仓库（git init）
> - 创建.gitignore文件（应排除.env、__pycache__/、*.log、test_results/等）
> - 添加所有文件（git add .）
> - 提交（git commit -m "first commit"）
> - 关联远程仓库（git remote add origin ...）
> - 推送（git push -u origin main）
>
> 请给出可以直接复制粘贴的命令，并解释每个命令的作用。

需要特别注意排除的文件：
- `.env`（包含API Key）
- `__pycache__/`（Python缓存目录）
- `*.log`（日志文件）
- `test_results/`（如果包含测试数据中的个人信息）
- 任何大于100MB的文件（视频文件可上传到云盘并放链接）

7.7.3 最终一致性检查

在推送之前，做一次“从零开始的完整性验证”：

1. 在一个新目录中克隆你的仓库（或重新下载代码）
2. 按照README的“安装”步骤配置环境
3. 按照“运行”步骤启动Gradio界面
4. 按照“快速开始”完成一次完整的分析
5. 访问API的`/docs`页面，测试一个端点

如果任何步骤失败，记录问题并修复，然后重新验证。这个过程会暴露你在开发中忽略的依赖缺失、路径错误或文档不准确。

7.8 本章小结

第七章完成了从“可运行的技能”到“可交付的产品”的关键跨越。我们围绕三个核心任务展开了系统性的方法：

第一，API封装。我们学习了如何从Skill中识别适合封装为API的子功能，应用四个筛选标准（输入输出明确、功能独立、高频潜力、性能可预测）进行评估。通过FastAPI框架和Vibe Coding，我们可以用自然语言描述API需求，生成完整的服务端代码、错误处理逻辑和自动文档。API封装的本质是将你的分析能力从“有人值守”转变为“无人值守、随时可被调用”的服务——这是工具从个人辅助升级为团队基础设施的关键一步。

第二，关键修复与文档。基于测试报告中的发现，我们建立了优先级驱动的修复策略，区分低成本高影响的问题（立即修复）与高成本低影响的问题（文档中标注已知限制）。同时，我们使用AI辅助生成了专业的README文档和演示视频脚本。文档的质量决定了他人是否能够理解和使用你的工具，而演示视频是你在最终展示中传达价值的核心载体。

第三，交付物整合。我们列出了完整的最终交付物清单，涵盖了代码、文档、测试报告、日志、视频和GitHub仓库。通过“从零开始的完整性验证”，确保所有文件相互一致、所有步骤可复现。

完成了第七章的工作，你的项目已经处于“可交付”状态。在第8章（最终展示与课程总结）中，你将有机会向全班和评审嘉宾展示你的成果，并参与关于“AI时代设计师新角色”的总结性讨论。在此之前，请利用最后一轮时间完成表格中的最后确认——推动那个勾选框从□变为✅。

思考与练习

一、概念理解题

1. 请解释为什么“把整个Skill做成一个API”是不好的设计。一个好的API候选应该满足哪些筛选标准？

2. FastAPI的`/docs`端点提供了什么价值？为什么说它“让API天然自带文档”？

3. 在API开发中，为什么需要将核心分析逻辑从Skill的GUI代码中提取出来？提取后如何进行独立性验证？

4. 基于测试报告的修复优先级矩阵中，什么类型的问题应该标记为P0（立即修复）？什么类型的问题适合放入README的“已知限制”？

二、分析应用题

5. 以下是一个API端点的错误响应设计，请指出其中的问题并提出改进方案。

   ```json
   {
     "error": "Internal Server Error",
     "detail": "list index out of range"
   }
   ```

6. 你的Skill测试报告显示：在“困难”级别的测试数据上准确率仅为55%，主要错误模式是将反讽误判为正面。测试报告中建议“优化提示词”。请写出两到三条Vibe Coding修改指令，指导AI改进提示词以解决这个问题。

三、实践与反思题

7. 基于你的Skill，完成以下API封装工作，并提交相关文件：
    - 从Skill代码中提取至少一个独立分析模块
    - 使用FastAPI将其封装为API端点
    - 编写API测试脚本，运行并通过至少5个测试用例
    - 提供客户端调用示例代码
8. 使用AI辅助生成README初稿，然后手工审核修改。提交最终的README.md文件和一份“审核修改记录”，列出你修改了AI生成的哪些内容以及修改的原因（至少3处）。
9. **个人反思**（500字）：在从Skill到API的封装过程中，你遇到的最大技术挑战是什么？你是如何通过与AI协作解决的？你认为“将设计研究方法封装为API”这一能力对设计师在AI时代的职业发展有什么价值？

参考文献

[1] FastAPI Documentation. (2025). https://fastapi.tiangolo.com

[2] Pydantic Documentation. (2025). https://docs.pydantic.dev

[3] Swagger UI. (2025). https://swagger.io/tools/swagger-ui/

[4] REST API Design Guidelines. (2024). Microsoft API Design.

[5] GitHub Documentation. (2025). https://docs.github.com

[6] Karpathy, A. (2024). Vibe coding: A new way to program. *Personal blog / X*.

本章引用统计数据：本章全文约 19,000 字。